Document no: P1995R1
Date: 2020-03-02
Authors: Joshua Berne, Timur Doumler, Andrzej Krzemieński, Ryan McDougall, Herb Sutter
Reply-to: jberne4@bloomberg.net
Audience: SG21

Contracts — Use Cases

Introduction

SG21 has gathered a large number of use cases for contracts between teh WG21 Cologne and Belfast meetings. This paper presents those use cases, along with some initial results from polling done of SG21 members to identify some level of important to the community for each individual use case.

Each use case has been assigned an identifier that can be used to reference these use cases in other papers, which will hopefully be stable. We expect this content to evolve in a number of ways:

- Long descriptions of these use cases (found at the bottom of this document) could all benefit from additional contributions.
- Future poll results, analysis of these poll results, and other analysis of this data will help guide group decisions on further SG21 progress.
- New and old contracts proposals, along with the current possibilities with no language changes at all, should be explored and measured in relation to how they satisfy these use cases.

Revision History

Differences with P1995R0:

- Changed key test.standardized -> teach.standardized
- Adjusted wording for clarity on: dev.reason.behaviorcontrol, int.control.build, int.control.runtime

Polling

Polling occurred through SurveyMonkey between the July 2019 WG21 Cologne meeting and the November 2019 WG21 Belfast meeting. Participants were asked to choose from three answers for each of the 195 different use cases:

- Must Have (Must = 2)
- Nice to Have (Nice = 1)
- Not important (Not = 0)

Participants were also able to not answer at all for a given question (N/A), and this happened for a small number of cases. The table of use cases below lists 4 columns with the total number of each response chosen. The score column is the average value of the responses amongst those who provided a response.

All Use Cases and Poll Results

Code	As A	In Order To	I Want To	N/A	Not	Nice	Must	Score
dev.reason.knowl	Developer	Reason explicitly	Annotate my program anywhere in the code with my current understanding of its structure or execution	0	0	10	20	1.66666
dev.reason.confidence	Developer	Reason explicitly	Express a spectrum of confidence in my annotations, from "unsure" and asking for validation, to "sure" and asking for some effect to be applied (eg. "maybe", "definitely", "assume" 'something')	0	10	10	10	1.00000
dev.reason.importance	Developer	Reason explicitly	Express a spectrum of importance of my annotations, from "critical" (eg. bring the system down) to "minor" (eg. lead to a slower fallback)	0	15	10	5	0.66666
dev.reason.cost	Developer	Reason explicitly	Express a spectrum of expected cost at compile or runtime of my annotations, from "unrunnable" to "expensive" to "cheap"	0	7	12	11	1.13333

dev.reason.behavior	Developer	Reason about executions	Have annotations affect the execution of my program in accordance with my expectations	0	7	9	14	1.233333
dev.reason.sideeffects	Developer	Reason about executions	Ensure annotations do not substantially change the meaning of my program whether enabled or disabled	0	3	8	19	1.533333
dev.reason.behaviorcontrol	Developer	Reason about executions	Have the effect of annotations on execution be user controllable (based on whatever aspects, if any, are available).	0	4	10	16	1.400000
dev.adapt	Developer	Adapt and progress with my project	Be able to easily change my confidence, importance, or other properties of my annotations over time	0	2	17	11	1.300000
dev.readable.syntax	Developer	Have readable annotations	Have annotations with a succinct and elegant syntax	0	0	13	17	1.566667
dev.readable.keywords	Developer	Have readable annotations	Have annotation keywords or names with intuitive, clear, and unambiguous meanings	0	0	11	19	1.633333
dev.readable.priority	Developer	Have readable annotations	Have my contract specification to be visually primary, and secondary information (syntax, hints, roles, levels, etc.) to not be distracting	0	7	17	6	0.966667
dev.parsable	Developer	Interoperate with tools or persons	A syntax that can both be parsed and can be reasoned about semantically	0	1	6	23	1.733333
dev.tooling	Developer	Interoperate with tools or persons	Expose annotations to tools that might leverage them (eg. code linter, static analyzer, semantic prover, compiler sanitizer, binary analyzer, code reviewer, etc.)	0	1	9	20	1.633333
cppdev.syntax.familiar	C++ Developer	Get up to speed	Have annotations use familiar syntax	0	12	15	3	0.700000
cppdev.syntax.cpp	C++ Developer	Get up to speed	Have annotations use C++ syntax	1	8	11	10	1.068966
cppdev.syntax.reuse	C++ Developer	Reuse code	Have annotations use my custom types or functions	0	1	9	20	1.633333
cppdev.location	C++ Developer	Have a single source of truth	Use same source file for both code and annotations	0	1	4	25	1.800000
cppdev.syntax.macros	C++ Developer	Support modern features	Minimize use of macros	0	5	18	7	1.066667
cppdev.modules	C++ Developer	Support modern features	Be interoperable with modules	0	0	6	24	1.800000
cppdev.coroutines	C++ Developer	Support modern features	Be interoperable with coroutines	0	3	11	16	1.433333
cppdev.concepts	C++ Developer	Support modern features	Be interoperable with concepts	0	3	9	18	1.500000
cppdev.existing.std	C++ Developer	Use the standard library in-contract	Codify existing exposition-only standard library requirements	0	5	18	7	1.066667
cppdev.debugger	C++ Developer	Use Debugger	Have runtime able to launch a debugger from an annotation if necessary	0	10	12	8	0.933333
cppdev.build.legacy	C++ Developer	Use existing build modes	Have annotations affect executions depending on my existing build modes (eg. Debug or Release modes in VS)	0	9	13	8	0.966667

cdev.contracts	C Developer	Write contracts on my functions	Specify contracts in a way standardizable as part of the C language	1	21	6	2	0.34482
cdev.identifiers	C Developer	Write contracts on my functions	Use contracts with macro-safe keywords that are reserved C names (i.e., _Pre, _Post, _Assert, etc.)	1	22	6	1	0.27586
cdev.violationhandler	C Developer	Write contracts on my functions	Have a common violation handler for both violated C and C++ contracts	1	16	11	2	0.51724
cdev.ignorable	C Developer	Write contracts on my functions	Make all contract semantics optional (so as not to change WG14-N2385 6.7.11 p2)	1	23	3	3	0.31034
ccppdev.interop	Mixed C/C++ Developer	Maintain mixed code base	Not lose contracts when crossing languages	1	12	14	3	0.68965
cdev.cppinterop	Mixed C/C++ Developer	Write contracts on my functions	Expose my contracts to C++ developers through 'extern "C"' declarations of my functions	1	16	12	1	0.48275
api.communicate.inputsoutputs	API Developer	Communicate my interface to users	Document the expected inputs and expected outputs on my interface	0	3	5	22	1.63333
api.establish.check	API Developer	Establish a contract	Have validation inform me which output values are unexpected or invalid	0	2	14	14	1.40000
api.establish.validate_invariants	API Developer	Establish a contract	Have validation inform me which class invariants are violated	0	2	14	14	1.40000
api.establish.values	API Developer	Establish a contract	Have validation inform user which input values are unexpected or invalid	0	1	11	18	1.56666
api.establish.preconditions	API Developer	Establish a contract	Have contracts specify their pre-conditions as logical predicates	0	2	3	25	1.76666
api.establish.invariants	API Developer	Establish a contract	Have contracts specify their class invariants as logical predicates	0	5	13	12	1.23333
api.establish.postconditions	API Developer	Establish a contract	Have contracts specify their post-conditions as logical predicates	0	5	5	20	1.50000
api.express.values	API Developer	Express predicates	Make reference to either the values of my inputs, or other in-scope identifiers	0	2	2	26	1.80000
api.establish.changedvalues	API Developer	Establish a contract	Make reference to the before and after values of in-out variables (ie. passed by pointer or reference) in post-conditions	0	7	11	12	1.16666
api.establish.changedmembers	API Developer	Establish a contract	Make reference to the before and after values of mutable class members (eg. *new_size = old_size+1* after push_back) in post-conditions	0	7	14	9	1.06666
api.establish.changedstate	API Developer	Establish a contract	Make reference to the before and after values of global state (eg., *global >= old(global) + 1*) in post-conditions	0	13	14	3	0.66666
api.extend.exceptionsafety	API Developer	Extend contractual aspects	Annotate operations as being exception safe	0	13	17	0	0.56666
api.extend.threadsafety	API Developer	Extend contractual aspects	Annotate operations as being thread safe	0	13	17	0	0.56666
api.extend.atomicity	API Developer	Extend contractual aspects	Annotate operations as being atomic (ie. all or no changes become visible)	0	15	15	0	0.50000

api.extend.realtime	API Developer	Extend contractual aspects	Annotate operations as real-time (ie. guaranteed to complete within a time frame)	0	20	9	1	0.366667
api.extend.determinism	API Developer	Extend contractual aspects	Annotate operations as being deterministic (ie. same outputs for same inputs)	0	11	13	6	0.833333
api.extend.purity	API Developer	Extend contractual aspects	Annotate operations as functionally pure (ie. no side effects)	0	10	15	5	0.833333
api.extend.sideeffects	API Developer	Extend contractual aspects	Annotate operations as having global side effects (ie. write to singleton, file, network, or database)	0	13	17	0	0.566667
api.extend.complexity	API Developer	Extend contractual aspects	Annotate algorithmic complexity	0	19	11	0	0.366667
api.express.runnability	API Developer	Express unrunnable contracts	Be able to use a predicate that is not evaluated at runtime, because it might be unsafe to run or have stateful side effects	0	6	14	10	1.133333
api.express.undefined	API Developer	Express unrunnable contracts	Be able to use a predicate that doesn't have a definition, because it hasn't been written yet, or is infeasible to run	0	8	12	10	1.066667
api.express.uncheckable	API Developer	Express uncheckable contracts	Be able to use a predicate that is not evaluated, because it is simply a semantic placeholder for a tool	0	8	12	10	1.066667
api.express.unimplementable	API Developer	Express uncheckable contracts	Be able to use a predicate that cannot have a complete definition, because it is inexpressible in the language	0	11	9	10	0.966667
api.establish.responsibility	API Developer	Establish responsibility boundaries	Inform users which errors are the responsibility of the caller, and which are the callee	0	4	11	15	1.366667
api.resp.preassert	API Developer	Establish responsibility boundaries	Annotate assertions inside function bodies that indirectly test preconditions (such as malformed data discovered while performing the algorithm) should be reported to the caller as precondition failures	0	12	13	5	0.766667
api.contract.interface	API Developer	Have contract as part of my interface	Declare contract when I declare the function	0	3	6	21	1.600000
api.contract.private	API Developer	Keep my user interfaces clean and narrow	Be able to access private implementation details of the class so I don't have to widen public interface to declare predicates	0	5	12	13	1.266667
api.contract.redeclaration	API Developer	Keep my public interfaces clean and concise	Place function contract conditions on any declaration (e.g., on redeclarations at the bottom of the header, or on the definition in an implementation file, where they are less distracting).	0	12	11	7	0.833333
api.contract.errorhandling	API Developer	Move contract violation out of error handling	Replace uses of error handling to express contract violation (eg. *operator[](size_t n) noexcept [[pre: n < size()]]* instead of throwing)	0	7	11	12	1.166667
cppapi.invariants	C++ API Developer	Write classes	Declare class invariants that all of my public functions need to maintain	1	4	13	12	1.275862
cppapi.class.preconditions	C++ API Developer	Maintain a class hierarchy	Ensure overriding methods have same or wider preconditions (see: Liskov substitution principle)	0	4	20	6	1.066667

cppapi.class.postconditions	C++ API Developer	Maintain a class hierarchy	Ensure overriding functions meet their base class postconditions when their base class preconditions are met (see: Liskov substitution principle)	0	2	20	8	1.200000
cppapi.class.variability	C++ API Developer	Maintain a class hierarchy.	Allow overriding functions to have narrower preconditions/wider postconditions if I want to	0	13	14	3	0.666667
api.class.publicinterface	C++ API Developer	Express public class invariants	Express a restriction on the public interface of a type that all callers of the type can depend upon: can mention only public members, and is checked on entry and exit from this type's code	0	9	19	2	0.766667
api.class.publicinvariants	C++ API Developer	Express public class invariants	Check invariants before and after every public method (when called from outside the type, not when one member function calls another)	0	5	21	4	0.966667
api.class.publiccalls	C++ API Developer	Express public class invariants	Check invariants before and after calling functions that are not part of this type (including virtual calls)	0	8	20	2	0.800000
api.class.baseinterface	C++ API Developer	Express base class invariants	Express a restriction on the protected interface of a type that derived types can depend upon: can mention only protected and public members, and is checked on entry and exit from this type's code	0	14	15	1	0.566667
api.class.baseinvariants	C++ API Developer	Express base class invariants	Check invariants on entry and exit of every protected method (when called from the derived type, not when one base member function calls another)	1	14	13	2	0.586207
api.class.basecalls	C++ API Developer	Express base class invariants	Check invariants before and after every call to a virtual function (when calling to the derived type)	0	10	18	2	0.733333
api.class.privateinterface	C++ API Developer	Express private class invariants	Express an internal restriction on the private implementation of a type, can mention any member, and is checked on entry and exit from this type's code	0	12	14	4	0.733333
api.class.privateinvariants	C++ API Developer	Express private class invariants	Check invariants on entry and exit of every public method (when called from outside the type, not when one member function calls another)	0	9	17	4	0.833333
api.class.privatecalls	C++ API Developer	Express private class invariants	Check invariants before and after calling functions that are not part of this type (including virtual calls)	0	13	15	2	0.633333
api.class.testing	C++ API Developer	Test my classes	For every member or friend function in my class, run my unit test framework with checking enabled for every assertion at the point where it is written, and check every postcondition at every non-exceptional exit, and test my class invariants on entry and exit from this type's code	0	10	9	11	1.033333
cppapi.contracts.async	C++ API Developer	Enforce contracts in async code	Express contracts on callbacks such as std::function, function pointers, or references to functions, lambdas, or function objects	0	4	21	5	1.033333
cppapi.contracts.exception	C++ API Developer	Enforce contracts in exception safe code	Express contracts on exceptional exit	0	11	13	6	0.833333
cppapi.variadic	C++ API Developer	Use contracts with variadic templates	Allow predicate (fold) expansion	0	4	15	11	1.233333
api.coroutines	C++ API Developer	Use coroutines	Define and check pre and post conditions as I would a regular function	0	4	15	11	1.233333

api.coroutines.invariants	C++ API Developer	Use coroutines	Define and check invariants over all entry and exit points from a coroutine (to its awaiter or promise)	0	7	19	4	0.900000
int.conform.violation	Integration Developer	Conform to a contract	Be informed any time an interface's contract is violated	0	2	12	16	1.466667
int.conform.postconditions	Integration Developer	Conform to a contract	Verify results from a call are expected output values	0	4	9	17	1.433333
int.build.headeronly	Integration Developer	Build multiple libraries	Use contract-enabled header-only libraries	0	0	6	24	1.800000
int.build.binaries	Integration Developer	Build multiple libraries	Use contract-enabled binary libraries	1	1	7	21	1.689655
int.build.binarycounts	Integration Developer	Build multiple libraries	Only be required to manage a small, common set of build/link configurations	0	6	15	9	1.100000
int.build.control	Integration Developer	Debug multiple libraries	Enable checks only within a selected library	0	1	14	15	1.466667
int.build.control2	Integration Developer	Debug multiple libraries	Enable checks on multiple libraries simultaneously	0	2	14	14	1.400000
int.debug.callsites	Integration Developer	Debug multiple call sites	Enable checks only on selected call sites	0	5	18	7	1.066667
int.violations.information	Integration Developer	Correct failed checks	Be informed what check failed, when, where, and how	0	1	9	20	1.633333
int.violations.transmit	Integration Developer	Correct failed checks	Transmit check failure information in environment-specific ways (logs, email, special hardware traps, popup windows, blazing sirens, etc).	0	5	14	11	1.200000
int.violations.custom	Integration Developer	Correct failed checks	Install custom violation handler where I can inject custom logic to trap errors	0	7	9	14	1.233333
int.violations.common	Integration Developer	Unify violation handling	Be able to override how library violations are handled in the combined software to point into my handling code	0	6	9	15	1.300000
int.violations.override	Integration Developer	Be independent of build environment	Be able to define and override violation handler via source code	1	14	11	4	0.655172
int.build.minimize	Integration Developer	Minimize checking overhead	Disable library postconditions, asserts, and invariants, without disabling library preconditions (assuming the library is tested and stable and my code is not)	0	8	14	8	1.000000
int.control.build	Integrated Software Provider	Ensure the combined software is correct	At build time, turn on and off what checking happens.	0	3	6	21	1.600000
int.control.runtime	Integrated Software Provider	At runtime, control what checking happens.	Turn checks on at run time	0	14	11	5	0.700000
int.conrol.subsets.build	Integrated Software Provider	Ensure the combined software is correct	Turn on any subset of individual (call site) checks on at build time	1	8	12	9	1.034483
int.control.subsets.runtime	Integrated Software Provider	Ensure the combined software is correct	Turn on any subset of individual (call site) checks on at run time	0	15	12	3	0.600000

int.consistency	Integrated Software Provider	Ensure the combined software is correct	Verify all annotations are globally consistent when integrated	1	6	20	3	0.896552
int.control.subsets	Integrated Software Provider	Ensure individual features are correct	Have a way to audit (named or semantic) subsets of checks for various deployments	1	9	16	4	0.827586
int.build.common	Integrated Software Provider	Manage binary delivery	Be able to use the same executable regardless of contract enforcement mode	0	16	10	4	0.600000
int.testing.control	Integrated Software Provider	Define "Code Under Test"	Selectively enable checking for a set of functions which could name either an individual function or an overload set	0	11	16	3	0.733333
int.testing.controltypes	Integrated Software Provider	Define "Code Under Test"	Selectively enable checking for a set of types and all their members	0	12	15	3	0.700000
int.testing.transitivity	Integrated Software Provider	Define "Code Under Test"	Selectively enable checking for a set of types and all their transitively nested types and members	0	13	14	3	0.666667
int.testing.modules	Integrated Software Provider	Define "Code Under Test"	Selectively enable checking for a translation unit or module and all (non transitive) types and functions within	0	6	19	5	0.966667
int.build.unchecked	Integrated Software Provider	Test final deliverable	Turn off build time checking to remove checking overhead	0	9	6	15	1.200000
int.runtime.unchecked	Integrated Software Provider	Test final deliverable	Turn off run time checking to remove checking overhead	0	3	5	22	1.633333
int.build.optimize	Integrated Software Provider	Test final deliverable	Turn on run time optimization to leverage annotation assumptions	0	6	12	12	1.200000
cpplib.headeronly	C++ Library Developer	Use templates	Be able to ship header only library	0	1	4	25	1.800000
cpplib.insulation	C++ Library Developer	Control the tradeoff between need for client recompilation and contract condition visibility	Insulate contract conditions with the function definition, or insulate only the definition while putting contract conditions on a redeclaration - visible to static analysis tools in all TUs.	0	11	13	6	0.833333
lib.maintenance.noconfig	Library Provider	Simplify maintenance	Not require extra build steps to be documented	0	14	10	6	0.733333
lib.maintenance.nowhining	Library Provider	Simplify maintenance	Not have users complain about my product due to modifications of annotations resulting from their build configuration	0	17	9	4	0.566667
lib.integration.noconfig	Library Provider	Support successful integration	Not require extra build steps to be learned or performed	0	13	13	4	0.700000
lib.integration.nowhining	Library Provider	Support successful integration	Not have my users accidentally modify my careful annotations	0	12	11	7	0.833333

arch.nomacros	Technical Architect	Maintain quality of code base	Express assertions in a way that does not rely on C macros (i.e., there is no valid technical reason for a programmer not to use the new way, including space, time, tooling, and usability/complexity reasons, compared to C's assert macro)	0	7	13	10	1.100000
arch.complete	Technical Architect	Have a consistent and holistic contracts facility	Specify preconditions/postconditions/assertions/invariants that express my expectations about the expected valid state of my program in the form of compilable boolean expressions, that can be checked statically or dynamically (as opposed to disjointed state where these features are factored into bits)	1	3	12	14	1.379310
hardware.performance	Hardware Architect	Improve system-level performance	Be able to design new hardware + optimizations, carefully dovetailed into one another, that depend on statically-unprovable facts being annotated in the code	2	8	10	10	1.071429
sdev.bestpractices	Senior Developer	Set an example	Demonstrate best practice in defensive programming	0	3	9	18	1.500000
sdev.quality	Senior Developer	Enforce code quality	Discourage reliance on observable out-of-contract behavior by causing check failure to hard stop program or build	0	8	6	16	1.266667
sdev.maturity	Senior Developer	Enforce mature, finalized contracts	Disable continuation on violation of stable and correct individual contracts	0	4	7	19	1.500000
sdev.control	Senior Developer	Enforce mature, finalized contracts	Disable remapping of semantics on stable and correct individual contracts	0	9	12	9	1.000000
jdev.understand.contracts	Junior Developer	Understand the API	A uniform, fluent description of expected input values, expected output values, side effects, and all logical pre and post conditions	0	1	15	14	1.433333
jdev.understand.violations	Junior Developer	Understand the API	Be informed when my usage is out of contract	0	0	9	21	1.700000
jdev.understand.buildfailures	Junior Developer	Understand the program	Know why my software is not building	0	3	6	21	1.600000
jdev.understand.aborting	Junior Developer	Understand the program	Know why my software is aborting	0	1	9	20	1.633333
jdev.understand.omniscience	Junior Developer	Understand the program	Know why my software is out of contract	0	1	11	18	1.566667
jdev.understand.buildviolation	Junior Developer	Understand the program	Know that my program or build was halted due to contract violation	0	0	7	23	1.766667
jdev.understand.all	Junior Developer	Understand the facility	Be able to build a program with contracts after reasonably short tutorial	0	4	12	14	1.333333
jdev.understand.keywords	Junior Developer	Understand the facility	Have keywords with precise and unambiguous meanings	0	2	13	15	1.433333
jdev.bestpractices	Junior Developer	Improve my code	Learn about software best practices by example	0	4	12	14	1.333333
adev.fast	Agile Developer	Iterate quickly	Be able to write and modify contracts quickly without heavy boiler plate or up front cost	0	1	14	15	1.466667
adev.evolve	Agile Developer	Safeguard evolving code	Assert against conditions I am aware of but not finished handling fully	0	8	12	10	1.066667

bdev.confidentiality	Business Developer	Maintain confidentiality	Not expose diagnostic information (source location, expressions, etc.) in the software I deliver to clients, even when I choose to have contracts enforced in the software I deliver	0	14	11	5	0.700000
pdev.speed	Performance Sensitive Developer	Enable better performance	Annotate my code with assumptions, likelihoods, or reachability information that a tool might not be able to deduce, but that I would be confident of	0	7	10	13	1.200000
pdev.morespeed	Performance Sensitive Developer	Enable better performance	Be able to give statically-unprovable facts to current and novel optimizers in terms of semantics my program does not depend-on but optimizers can't figure out	0	6	11	13	1.233333
pdev.footgun	Performance Sensitive Developer	Enable better performance	Accept responsibility for a malformed program that might result from eventually false information given by my annotations	0	7	8	15	1.266667
pdev.safety.isolation	Performance Sensitive Developer	Have safety critical paths	Isolate safety checks from performance annotations	0	7	11	12	1.166667
pdev.safety.critical	Performance Sensitive Developer	Have safety critical paths	Retain checking even when optimizing with performance annotations	0	8	8	14	1.200000
qdev.checkall	Quality Sensitive Developer	Enable full checking	Ensure all checks (pre, post, assert, invariant) are enabled	0	2	10	18	1.533333
qdev.correctness	Quality Sensitive Developer	Validate correctness	Signify the predicates that should be verified by an analysis tool	0	6	19	5	0.966667
qdev.tooling	Quality Sensitive Developer	Manage multiple tools	Signify subset of individual annotations to be consumed by a specific kind of verification tool	0	14	14	2	0.600000
qdev.tooling.control	Quality Sensitive Developer	Manage multiple tools	Signify subset of individual annotations to be consumed by a specific instance of verification tool	0	14	15	1	0.566667
qdev.tooling.undefined	Quality Sensitive Developer	Manage multiple tools	Use predicates that may not be understood by all instances of verification	0	9	12	9	1.000000
qdev.tooling.undefinedkinds	Quality Sensitive Developer	Manage multiple tools	Use predicates that may not be understood by all kinds of verification	0	8	12	10	1.066667
qdev.tooling.behavior	Quality Sensitive Developer	Manage multiple tools	Integrate the results of that static checker into how my program behaves in different ways: assume proven predicates, make unprovable predicates ill- formed, etc.	0	15	11	4	0.633333
qdev.testing	Quality Sensitive Developer	Unit test predicates	Override failure handler to trigger test failure instead of termination	0	8	9	13	1.166667
qdev.handler.testing	Quality Sensitive Developer	Unit test violation handlers	Have a way to run handler on all combinations of available build modes	0	11	13	6	0.833333
qdev.fuzz.testing	Quality Sensitive Developer	Catch unexpected failure modes	Log all predicate failure during fuzz testing	0	5	14	11	1.200000
crit.control	Critical Software Developer	Have a verifiable release system	Be able to control the configuration of contracts from a central point	0	4	15	11	1.233333

crit.noundef	Critical Software Developer	Avoid undefined behavior	Have contract violation at run-time always have well-defined behavior	0	17	4	9	0.733333
crit.recovery	Critical Software Developer	Not have a faulty program lead to catastrophic failure	Have access to a recovery path after contract violation	0	15	8	7	0.733333
crit.redundancy	Critical Software Developer	Not have a faulty program lead to catastrophic failure	Be able to express error handling that may be redundant with contract checking	0	11	8	11	1.000000
crit.interaction	Critical Software Developer	Not have a faulty program lead to catastrophic failure	Not have contract build or run modes possibly be able to change or disable related error handling in any way	0	13	5	12	0.966667
crit.locality	Critical Software Developer	Be assured a critical violation uses a critical recovery path	Couple recovery path to a specific contract within the source	0	19	6	5	0.533333
crit.testing	Critical Software Developer	Meet code coverage requirements	Be able to run both success and failure branches in my test environment	0	8	11	11	1.100000
crit.production.checking	Critical Software Developer	Have redundant layering	Be able to continue to run checks in a production environment (even after formal testing is complete)	1	6	8	15	1.310345
crit.more.coverage	Critical Software Developer	Maximize coverage	Be able to run checks in a production environment that are considered "cheap" compared to the expected cost of entering an invalid state	0	6	13	11	1.166667
crit.noassume	Critical Software Developer	Avoid unexpected or undefined behavior	Ensure checks will never be __assume'd/__builtin_assume'd by the compiler as if they were facts injected into the program (otherwise, if such an assumption ever failed, I would be running a different program that is not equivalent to the one I wrote; assumptions can expand the set of possible executions by injecting facts not otherwise knowable to the compiler)	0	13	5	12	0.966667
sec.noattacks	Security Sensitive Developer	Limit attack vectors	Be unable to insert code paths (eg. violation handlers) at run time (eg. build time only)	1	8	9	12	1.137931
sec.certify	Security Sensitive Developer	Deliver a certified product	Have build tool only link to a preapproved violation handler	0	12	15	3	0.700000
analysis.runtime	User of Analysis Tools	Improve runtime correctness	Have runtime checks generated by the tool	1	8	14	7	0.965517
analysis.optimization	User of Analysis Tools	Improve runtime performance	Have runtime optimizations generated by the tool	1	7	17	5	0.931034
analysis.symbolic	User of Analysis Tools	Allow symbolic analysis	Have symbolic proofs for soundness and consistency performed before compile time	1	5	20	4	0.965517
analysis.compiletime	User of Analysis	Allow code analysis	Have code source, AST, or instruction inspection during compile time	1	8	17	4	0.862069

	Tools							
analysis.binaries	User of Analysis Tools	Allow binary analysis	Have binary inspection after compile time	1	17	10	2	0.48275
analysis.information	User of Analysis Tools	Improve the quality of analysis	Be able to hint to the analyzer information it may be unable to deduce from source code alone (eg. *5 / opaque(); [[opaque() != 0]]*)	1	6	17	6	1.00000
analysis.legacy	Provider of Analysis Tools	Extend my existing engine	Be able to map pre-existing contract features in tools to a standardized language syntax	1	14	10	5	0.68965
teach.bestpractices	Teacher	Demonstrate best practice	Be able to express defensive programming, programming by contract, and test driven development to introductory students	0	2	13	15	1.43333
teach.standardized	Teacher	Demonstrate best practice	Not rely on custom libraries or proprietary extensions	0	3	12	15	1.40000
teach.lifecycle	Teacher	Demonstrate best practice	Demonstrate mock lifecycle by switching simple compiler flags to control which checks are enabled	0	5	19	6	1.03333
teach.portable	Teacher	Manage many students	Have examples compilable by a standard compiler on any system	0	4	3	23	1.63333
teach.dumbstudents	Teacher	Manage many students	Have examples that are easy to build without digression into build systems	0	6	9	15	1.30000
teach.teachable	Teacher	Build layers of understanding	Have simple explanation of assertions and their use to support simple programming tasks, including debugging erroneous programs.	0	3	9	18	1.50000
teach.layering	Teacher	Build layers of understanding	Support the ability for advanced uses of contracts to be distributed across many different courses in a C++-focused computer science curriculum.	0	6	15	9	1.10000
compiler.benice	Compiler Developer	Deliver best experience to my customers	Maximize implementation freedom by limiting what is strictly required by the standard	0	12	12	6	0.80000
compiler.best	Compiler Developer	Deliver the best implementation	Have a clear and simple specification that meets clear need	0	1	10	19	1.60000
large.complex	Large Codebase Developer	Debug complex issues	Have composable and fine grained control over which checks are run, without requiring source code changes. Specifically the checks for only one function or some grouping of functions	0	5	14	11	1.20000
large.critical	Large Codebase Developer	Enable/Disable checking on critical/hot paths	Control whether checks are run based on where they are being called from	0	9	13	8	0.96666
large.modernize	Large Codebase Owner	Modernize my code base	Introduce standardized contracts to replace my macro-based contracts	0	1	12	17	1.53333
large.stillmacros	Large Codebase Owner	Modernize my code base	Have my existing macro-based facilities interoperate smoothly with standardized contracts so I can do the migration gradually	0	5	16	9	1.13333
large.observation	Large Codebase Owner	Introduce new contracts into an existing system	Have failed individual checks from existing code optionally warn instead of hard stop	0	7	9	14	1.23333
large.introduction	Large Codebase Owner	Introduce new contracts into	Have failed checks from a new library optionally warn instead of hard stop	0	7	9	14	1.23333

		an existing system						
large.separability	Large Codebase Owner	Introduce new parameters or invariants into a contracts based system	Be able to include distinct clauses for each parameter or invariant with their own individual failure or build controls	0	17	8	5	0.600000
large.newenvironment	Large Codebase Owner	Introduce new elements into a contracts based system	Have failed checks caused by a change in environment optionally warn instead of hard stop	0	10	13	7	0.900000
large.newcompiler	Large Codebase Owner	Introduce new elements into a contracts based system	Have failed checks caused by a change in compiler optionally warn instead of hard stop	0	10	15	5	0.833333
large.nogoingback	Large Codebase Owner	Prevent regressions	Have trusted contracts fail fast and hard stop	0	2	11	17	1.500000
large.scalability	Large Codebase Owner	Scale violation handling	Be able to log violations in my organization specific format	0	5	11	14	1.300000
large.simulation.disable	Large Codebase Owner	Allow simulation or post-mortem testing of known failure modes	Optionally disable checking on a subset of individual annotations	0	10	10	10	1.000000
large.simulation.enable	Large Codebase Owner	Allow simulation or post-mortem testing of known failure modes	Optionally allow checking of a subset of individual annotations to fail and access its recovery path	0	13	9	8	0.833333
large.simulation.ignore	Large Codebase Owner	Allow simulation or post-mortem testing of known failure modes	Optionally allow checking of a subset of individual annotations to fail and continue failing	0	14	9	7	0.766667
large.perfcontrol.build	Large Codebase Owner	Manage performance cost	Constrain the set of built time checks according to their performance overhead	0	13	12	5	0.733333
large.perfcontrol.runtime	Large Codebase Owner	Manage performance cost	Constrain the set of runtime checks according to their performance overhead	0	6	11	13	1.233333
large.narrowing	Large Codebase Owner	Tune contract width in complex system	Be able to narrow individual contract so it fails in testing not in production	0	11	13	6	0.833333
embedded.nochecking	Small Machine Developer	Minimize executable footprint	Remove all checking and diagnostic (eg. source location) overhead entirely from the final binary	0	4	7	19	1.500000
embedded.nologging	Small Machine Developer	Minimize executable footprint	Remove all logging and diagnostic (but not checking) overhead from the final binary	1	4	14	11	1.241379
embedded.minimize	Small Machine	Minimize executable	Remove all but the most important diagnostic overhead from the final binary	0	7	14	9	1.066667

	Developer	footprint						
wg21.everythingelse	Language Developer	Interoperate with Contracts	Have a clear way to understand how contracts will interact with the standard library	1	4	11	14	1.344828
wg21.otherfeatures	Language Developer	Extend contracts beyond pre/post conditions on functions	Be able to use contract-like syntax on past or present runtime checkable language features such as switches, pattern matching, etc. or what might happen on signed integer overflow, etc. This might allow configuration of trapping, logging, or assuming in other areas of language UB.	0	12	14	4	0.733333

Detailed Descriptions

dev.reason.knowl

As a Developer
In order to Reason explicitly
I want to Annotate my program anywhere in the code with my current understanding of its structure or execution

Must Have: 20, *Nice to Have:* 10, *Not Important:* 0

This general use case expresses the desire to place information about a program's expected execution state in many different places throughout the program - possibly including "upon function entry", "whenever this line of code is executed", "at all times when a class of this type is not actively executing a member function", or others.

dev.reason.confidence

As a Developer
In order to Reason explicitly
I want to Express a spectrum of confidence in my annotations, from "unsure" and asking for validation, to "sure" and asking for some effect to be applied (eg. "maybe", "definitely", "assume" 'something')

Must Have: 10, *Nice to Have:* 10, *Not Important:* 10

This high level use case expresses the need to attach information to contract conditions that is beyond the statement of the condition itself, and instead has user-provided metadata about both the confidence in the condition and the desired behavior of the program in relation to that condition.

dev.reason.importance

As a Developer
In order to Reason explicitly
I want to Express a spectrum of importance of my annotations, from "critical" (eg. bring the system down) to "minor" (eg. lead to a slower fallback)

Must Have: 5, *Nice to Have:* 10, *Not Important:* 15

This high level use case expresses the desire to have metadata on contract conditions associated with what downsides violations might have, perhaps indicating that while a certain condition is expected to be met, the library does guarantee that the downsides will not be catastrophic.

dev.reason.cost

As a Developer
In order to Reason explicitly
I want to Express a spectrum of expected cost at compile or runtime of my annotations, from "unrunnable" to "expensive" to "cheap"

Must Have: 11, *Nice to Have:* 12, *Not Important:* 7

This use case expresses the desire to have metadata about contract conditions that capture at least 3 (if not more) granularities of "cost" to be used as input in some way to other decisions about what the contracts might do or how they might be interpreted.

dev.reason.behavior

As a Developer
In order to Reason about executions
I want to Have annotations affect the execution of my program in accordance with my expectations

Must Have: 14, *Nice to Have:* 9, *Not Important:* 7

The desire to have behaviors (such as generating runtime checks or optimizations) and the expected behavior explicitly defined by the standard in order to help users reason about what a program will do/has done when a contract is violated. Note that having this in the standard allows for leveraging expectations of behavior across all compliant platforms.

dev.reason.sideeffects

As a Developer
In order to Reason about executions
I want to Ensure annotations do not substantially change the meaning of my program whether enabled or disabled

Must Have: 19, *Nice to Have:* 8, *Not Important:* 3

This high-level use case expresses a desire that any execution effective transformations an annotation might apply be "reasonable", ie. not surprising or counter intuitive (such as time-travel optimizations).

There is a tension between allowing side effects in contract conditions and disallowing them completely. In general, writing code with absolutely no side effects is very hard, and there are pitfalls if the language is actively hostile to accidental side effects. On the other hand, conditions with side effects are also not elidable by the compiler, since the act of checking them is observable.

- N4810 contracts made any side effect undefined behavior.
- P1670 suggested allowing side effects but also, when contracts are runtime checked, allowing elision of the predicate (along with its side effect) if the predicate result can be determined.

dev.reason.behaviorcontrol

As a Developer
In order to Reason about executions
I want to Have the effect of annotations on execution be user controllable (based on whatever aspects, if any, are available).

Must Have: 16, *Nice to Have:* 10, *Not Important:* 4

This encompasses some specific need to control, either when writing code or when building code, what behavior (if any) is associated with a contract condition.

dev.adapt

As a Developer
In order to Adapt and progress with my project
I want to Be able to easily change my confidence, importance, or other properties of my annotations over time

Must Have: 11, *Nice to Have:* 17, *Not Important:* 2

dev.readable.syntax

As a Developer
In order to Have readable annotations
I want to Have annotations with a succinct and elegant syntax

Must Have: 17, *Nice to Have:* 13, *Not Important:* 0

dev.readable.keywords

As a Developer
In order to Have readable annotations
I want to Have annotation keywords or names with intuitive, clear, and unambiguous meanings

Must Have: 19, *Nice to Have:* 11, *Not Important:* 0

Any keywords chosen for use within the feature should be easily distinguishable (to avoid name churn such as expects/ensures to pre/post) and be very clearly matched to what they will do within the language (unlike, for example, axiom).

dev.readable.priority

As a Developer
In order to Have readable annotations
I want to Have my contract specification to be visually primary, and secondary information (syntax, hints, roles, levels, etc.) to not be distracting

Must Have: 6, *Nice to Have:* 17, *Not Important:* 7

This use case indicates a preference to have any metadata about a contract be visually very minimal or come after the predicate itself.

dev.parsable

As a Developer
In order to Interoperate with tools or persons
I want to A syntax that can both be parsed and can be reasoned about semantically

Must Have: 23, *Nice to Have:* 6, *Not Important:* 1

dev.tooling

As a Developer
In order to Interoperate with tools or persons
I want to Expose annotations to tools that might leverage them (eg. code linter, static analyzer, semantic prover, compiler sanitizer, binary analyzer, code reviewer, etc.)

Must Have: 20, *Nice to Have:* 9, *Not Important:* 1

cppdev.syntax.familiar

As a C++ Developer
In order to Get up to speed
I want to Have annotations use familiar syntax

Must Have: 3, *Nice to Have:* 15, *Not Important:* 12

cppdev.syntax.cpp

As a C++ Developer
In order to Get up to speed
I want to Have annotations use C++ syntax

Must Have: 10, *Nice to Have:* 11, *Not Important:* 8 , *No Answer:* 1

cppdev.syntax.reuse

As a C++ Developer
In order to Reuse code
I want to Have annotations use my custom types or functions

Must Have: 20, *Nice to Have:* 9, *Not Important:* 1

Rather than requiring special functions or types (or some completely new thing), contract should be able to leverage any program logic related to their statement that is already written/writable in C++.

cppdev.location

As a C++ Developer
In order to Have a single source of truth
I want to Use same source file for both code and annotations

Must Have: 25, *Nice to Have:* 4, *Not Important:* 1

This as a preference to needing to specify contracts in a completely separate metadata file of some sort.

cppdev.syntax.macros

As a C++ Developer
In order to Support modern features
I want to Minimize use of macros

Must Have: 7, *Nice to Have:* 18, *Not Important:* 5

The desire is to not require use of macros in order to satisfy the majority of use cases.

cppdev.modules

As a C++ Developer
In order to Support modern features
I want to Be interoperable with modules

Must Have: 24, *Nice to Have:* 6, *Not Important:* 0

All contract control features and behaviors should be interoperable in a reasonable way with any partially or fully modularized C++ program.

cppdev.coroutines

As a C++ Developer
In order to Support modern features
I want to Be interoperable with coroutines

Must Have: 16, *Nice to Have:* 11, *Not Important:* 3

Contracts on coroutines open up a number of new situations to consider because there might be requirements on what the state of the program is whenever a coroutine resumes execution, and there might be promises a coroutine makes prior to each time it suspends.

cppdev.concepts

As a C++ Developer
In order to Support modern features
I want to Be interoperable with concepts

Must Have: 18, *Nice to Have:* 9, *Not Important:* 3

cppdev.existing.std

As a C++ Developer
In order to Use the standard library in-contract
I want to Codify existing exposition-only standard library requirements

Must Have: 7, *Nice to Have:* 18, *Not Important:* 5

In some shape or form, anything documented in the contracts of the standard library's functions (preconditions, postconditions, other behaviors) is a candidate for something that should be codifiable as a contract condition.

cppdev.debugger

As a C++ Developer
In order to Use Debugger
I want to Have runtime able to launch a debugger from an annotation if necessary

Must Have: 8, *Nice to Have:* 12, *Not Important:* 10

When handling a contract violation, the ability to trigger a debugger when possible is currently not standardized but should be available.

cppdev.build.legacy

As a C++ Developer
In order to Use existing build modes
I want to Have annotations affect executions depending on my existing build modes (eg. Debug or Release modes in VS)

Must Have: 8, *Nice to Have:* 13, *Not Important:* 9

Note that "debug" and "release" are not standardized things, but nothing specified in the standard should preclude those from having some impact on what behaviors contracts take on in those two modes. It should be possible for a vendor to map contract modes to their existing native modes.

cdev.contracts

As a C Developer
In order to Write contracts on my functions
I want to Specify contracts in a way standardizable as part of the C language

Must Have: 2, *Nice to Have:* 6, *Not Important:* 21 , *No Answer:* 1

Important considerations for C are related to any contracts that can go on a normal C function. C does have attributes, but it also explicitly calls out that a conforming implementation can ignore all attributes, as opposed to C++ which has made that a commonly held assumption but has not actually put wording in the standard to that effect.

cdev.identifiers

As a C Developer
In order to Write contracts on my functions
I want to Use contracts with macro-safe keywords that are reserved C names (i.e., _Pre, _Post, _Assert, etc.)

Must Have: 1, *Nice to Have:* 6, *Not Important:* 22 , *No Answer:* 1

Adding new identifiers with meaning in C is generally not acceptable for standardization, so compatible contracts would either need to use no contracts or support alternate reserved words for use with contracts.

cdev.violationhandler

As a C Developer
In order to Write contracts on my functions
I want to Have a common violation handler for both violated C and C++ contracts

Must Have: 2, *Nice to Have:* 11, *Not Important:* 16 , *No Answer:* 1

Importantly, satisfying this requirement would mean making the the argument to a violation handler meaningful in both C and C++, or requiring platform vendors to shim between the two (in the case of a C violation handler receiving a violation from C++ code).

cdev.ignorable

As a C Developer
In order to Write contracts on my functions
I want to Make all contract semantics optional (so as not to change WG14-N2385 6.7.11 p2)

Must Have: 3, *Nice to Have:* 3, *Not Important:* 23 , *No Answer:* 1

Assuming contracts continue to be rendered as attributes, C standardization would require they be semantically optional.

Note that, just as with C++, the ':' in the previous contract syntax does not match the grammar for attributes in either language, so by a strict reading of the standards there is no obligation to be ignorable. ('[[a:b]]' is not a valid attribute and should be diagnosed as invalid on any current C or C++ compiler). Many have expressed the view that this opinion is pedantic and that the spirit of the law is that anything between [[]]s should be ignorable.

Additionally, it is currently a conforming extension to throw away all tokens between a pair of [[]]s, and there exist numerous compilers that take advantage of that fact which would be broken by requiring behavior of any constructs that look like an attribute.

ccppdev.interop

As a Mixed C/C++ Developer
In order to Maintain mixed code base
I want to Not lose contracts when crossing languages

Must Have: 3, *Nice to Have:* 14, *Not Important:* 12 , *No Answer:* 1

cdev.cppinterop

As a Mixed C/C++ Developer
In order to Write contracts on my functions
I want to Expose my contracts to C++ developers through 'extern "C"' declarations of my functions

Must Have: 1, *Nice to Have:* 12, *Not Important:* 16 , *No Answer:* 1

api.communicate.inputsoutputs

As a API Developer
In order to Communicate my interface to users
I want to Document the expected inputs and expected outputs on my interface

Must Have: 22, *Nice to Have:* 5, *Not Important:* 3

api.establish.check

As a API Developer
In order to Establish a contract
I want to Have validation inform me which output values are unexpected or invalid

Must Have: 14, *Nice to Have:* 14, *Not Important:* 2

api.establish.validate_invariants

As a API Developer
In order to Establish a contract
I want to Have validation inform me which class invariants are violated

Must Have: 14, *Nice to Have:* 14, *Not Important:* 2

Class invariants have historically been considered of general use, but the performance impact of checking them can be surprisingly huge. Previously these have been left out of proposals for C++ to be added in at a later date.

api.establish.values

As a API Developer
In order to Establish a contract
I want to Have validation inform user which input values are unexpected or invalid

Must Have: 18, *Nice to Have:* 11, *Not Important:* 1

This establishes the basic requirement on preconditions in terms of input values.

The more detail available to users about how a contract condition has been violated the more useful they become. This means each specific condition is benefited by being separate (eg. 'x != 0' and 'y != 0' as distinct conditions instead of requiring that users use 'x != 0 && y != 0').

In addition to that, anything that might be able to capture those values and expose them to violation handlers for logging would again help benefit problem diagnosis.

api.establish.preconditions

As a API Developer
In order to Establish a contract
I want to Have contracts specify their pre-conditions as logical predicates

Must Have: 25, *Nice to Have:* 3, *Not Important:* 2

This establishes the basic requirement to use predicates to evaluate input values.

api.establish.invariants

As a API Developer
In order to Establish a contract
I want to Have contracts specify their class invariants as logical predicates

Must Have: 12, *Nice to Have:* 13, *Not Important:* 5

This establishes the basic requirement for invariants in terms of predicates on class members.

api.establish.postconditions

As a API Developer
In order to Establish a contract
I want to Have contracts specify their post-conditions as logical predicates

Must Have: 20, *Nice to Have:* 5, *Not Important:* 5

This establishes the basic requirement to use predicates to evaluate return values.

api.express.values

As a API Developer
In order to Express predicates
I want to Make reference to either the values of my inputs, or other in-scope identifiers

Must Have: 26, *Nice to Have:* 2, *Not Important:* 2

This establishes the basic requirement to reference in-scope variables in order capture values and compute predicates.

api.establish.changedvalues

As a API Developer
In order to Establish a contract
I want to Make reference to the before and after values of in-out variables (ie. passed by pointer or reference) in post-conditions

Must Have: 12, *Nice to Have:* 11, *Not Important:* 7

Postconditions often need to state things about how values of output parameters have changed, or how the values of other global state might have changed as a result of a function call. The ability to store copies of state from before the function call and reference that in a postcondition predicate will enable a wider variety of conditions to be formulatable.

Ada has this functionality builtin by providing ways to reference explicitly the original value of a function parameter. C++ makes this more complicated with the need to consider the handling of move-only or generally non-copyable types. More importantly, such copying should absolutely not happen if a contract is not being evaluated at runtime.

api.establish.changedmembers

As a API Developer
In order to Establish a contract
I want to Make reference to the before and after values of mutable class members (eg. *new_size = old_size+1* after push_back) in post-conditions

Must Have: 9, *Nice to Have:* 14, *Not Important:* 7

Capturing the pre-function state of member variables might also be needed to state the contracts on many member functions.

api.establish.changedstate

As a API Developer
In order to Establish a contract
I want to Make reference to the before and after values of global state (eg., *global >= old(global) + 1*) in post-conditions

Must Have: 3, *Nice to Have:* 14, *Not Important:* 13

Capturing arbitrary global state for use in a postcondition might prove useful.

This extends to the state of arbitrary other expressions and how that might change due to the invocation of a function. Consider this example of what might be a postcondition of a typical `sleep` function:

```
old(now()) <= now() + sleep_time
```

api.extend.exceptionsafety

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as being exception safe

Must Have: 0, *Nice to Have:* 17, *Not Important:* 13

Exception safety guarantees are often part of the English language contract of a function, and being able to state that (in a way that tools might be able to then pick up on and verify) would be useful.

api.extend.threadsafety

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as being thread safe

Must Have: 0, *Nice to Have:* 17, *Not Important:* 13

The proper use of a type in a multithreaded environment can benefit greatly from being documented in a way that can then be checked and verified.

api.extend.atomicity

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as being atomic (ie. all or no changes become visible)

Must Have: 0, *Nice to Have:* 15, *Not Important:* 15

Establishing more details about what might happen on failure, such as what external state might be changed in a remote database, is another useful condition to state.

api.extend.realtime

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as real-time (ie. guaranteed to complete within a time frame)

Must Have: 1, *Nice to Have:* 9, *Not Important:* 20

api.extend.determinism

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as being deterministic (ie. same outputs for same inputs)

Must Have: 6, *Nice to Have:* 13, *Not Important:* 11

api.extend.purity

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as functionally pure (ie. no side effects)

Must Have: 5, *Nice to Have:* 15, *Not Important:* 10

api.extend.sideeffects

As a API Developer
In order to Extend contractual aspects
I want to Annotate operations as having global side effects (ie. write to singleton, file, network, or database)

Must Have: 0, *Nice to Have:* 17, *Not Important:* 13

api.extend.complexity

As a API Developer
In order to Extend contractual aspects
I want to Annotate algorithmic complexity

Must Have: 0, *Nice to Have:* 11, *Not Important:* 19

api.express.runnability

As a API Developer
In order to Express unrunnable contracts
I want to Be able to use a predicate that is not evaluated at runtime, because it might be unsafe to run or have stateful side effects

Must Have: 10, *Nice to Have:* 14, *Not Important:* 6

Numerous predicates that might be useful to state also might change the state of a program in a paradoxical way if checked. The common example (originally put forward in P0380) is validating available items on an input iterator. Other important examples include checks that might require accessing special hardware, or doing extensive computations that would invalidate the results just by taking the time to do them.

These predicates are still useful for those reading the code, still might benefit from being validated against postconditions of other functions, and might provide some identifiable code improvements when analyzed by the compiler without actually executing them at runtime.

api.express.undefined

As a API Developer
In order to Express unrunnable contracts
I want to Be able to use a predicate that doesn't have a definition, because it hasn't been written yet, or is infeasible to run

Must Have: 10, *Nice to Have:* 12, *Not Important:* 8

Often during development it helps to write contracts first, and that might even predate having enough of an implementation to fully define the conditions. Having them in code ensures that the placeholder API to check them is maintained. Some functions may never be implementable due to time constraints, while others will eventually be filled in as time allows.

api.express.uncheckable

As a API Developer
In order to Express uncheckable contracts
I want to Be able to use a predicate that is not evaluated, because it is simply a semantic placeholder for a tool

Must Have: 10, *Nice to Have:* 12, *Not Important:* 8

Some predicates might even have no meaning within the language itself, but benefit other tools by being placed into the same contract framework as other predicates, with meaning to those external tools.

api.express.unimplementable

As a API Developer
In order to Express uncheckable contracts
I want to Be able to use a predicate that cannot have a complete definition, because it is inexpressible in the language

Must Have: 10, *Nice to Have:* 9, *Not Important:* 11

Many external static analysis tools, and even compilers, can check that some state is being handled properly which cannot be properly validated within the language itself. Consider functions such as "is_deletable" or "is_reachable".

api.establish.responsibility

As a API Developer
In order to Establish responsibility boundaries
I want to Inform users which errors are the responsibility of the caller, and which are the callee

Must Have: 15, *Nice to Have:* 11, *Not Important:* 4

One common realization with having checked contracts in the language is that they will then identify bugs that exist in programs. Identifying the source of those bugs swiftly is important, and for any contract that is dependent on a function being called properly that responsibility does not usually lie with the line of code where the contract is written, but with the place where the function has been called.

api.resp.preassert

As a API Developer
In order to Establish responsibility boundaries
I want to Annotate assertions inside function bodies that indirectly test preconditions (such as malformed data discovered while performing the algorithm) should be reported to the caller as precondition failures

Must Have: 5, *Nice to Have:* 13, *Not Important:* 12

Some function preconditions can be expressed as c++ expressions, but are runtime-expensive. However, if expressed inside function body as assertions after some function logic has already been executed, the check whether the function has everything that is required is cheap. If I use an assertion like this, I want to annotate this assertion as being "precondition-like". The semantics would be that if it fails, it is reported as "caller's bug" rather than callee's bug, and if the build is configured to runtime-check only preconditions, such assertion should also be runtime-checked.

As an example, consider a binary search that wishes to check that the input list is sorted. There is an expensive check (O(n)) that can be done to verify that the whole list is sorted on each call, but this would negate the performance benefits of doing a binary search to begin with. Another option is to check just that the elements adjacent to those that are visited are in properly sorted order. This will provide some chance of identify unsorted inputs without making performance unusable, but is also most cleanly done during the execution of the search algorithm. Unlike a normal in-body assertion, a violation identified by this kind of check should be reported to the caller, as it is a failure on the calling code's part to provide a properly sorted input list.

api.contract.interface

As a API Developer
In order to Have contract as part of my interface
I want to Declare contract when I declare the function

Must Have: 21, *Nice to Have:* 6, *Not Important:* 3

Putting contracts on the first function declaration makes them easily visible to anyone looking at a header file for understanding an API.

api.contract.private

As a API Developer
In order to Keep my user interfaces clean and narrow
I want to Be able to access private implementation details of the class so I don't have to widen public interface to declare predicates

Must Have: 13, *Nice to Have:* 12, *Not Important:* 5

Often, an interface might be very narrow - consider the interface on an iterator, which generally just has a few operators and no other functions - yet there might be ways to check parts of a narrow contract that depend on internal state. Exposing this state so that it can be checked publicly would require widening the API interface, which is counter to the desire to keep the use of a type as simple as possible.

On a similar note, sometimes the internal state only approximates the actual contract, and while better than no checking there is a desire to not expose that data publicly so that there is no assumption that the data represents the complete set of requirements on calling code.

api.contract.redeclaration

As a API Developer
In order to Keep my public interfaces clean and concise
I want to Place function contract conditions on any declaration (e.g., on redeclarations at the bottom of the header, or on the definition in an implementation file, where they are less distracting).

Must Have: 7, *Nice to Have:* 11, *Not Important:* 12

Many development methodologies revolve around making readable, well-documented header files to provide a way to understand API usage. Some contracts might be simple and readable, but others might involve complex conditions that distract from readability, and match very simple to read and understand prose contracts in the documentation.

The benefits (checkability, analysis, etc.) of having the encoded contracts should not come at the cost of making the primary visible public interface (the header file) more unreadable.

api.contract.errorhandling

As a API Developer
In order to Move contract violation out of error handling
I want to Replace uses of error handling to express contract violation (eg. *operator[](size_t n) noexcept [[pre: n < size()]]* instead of throwing)

Must Have: 12, *Nice to Have:* 11, *Not Important:* 7

Historically, rather than providing functions with narrow contracts, software might have been written to report misuse through exceptions. Contracts should provide a better alternative than giving all functions a fully wide contract.

cppapi.invariants

As a C++ API Developer
In order to Write classes
I want to Declare class invariants that all of my public functions need to maintain

Must Have: 12, *Nice to Have:* 13, *Not Important:* 4 , *No Answer:* 1

cppapi.class.preconditions

As a C++ API Developer
In order to Maintain a class hierarchy
I want to Ensure overriding methods have same or wider preconditions (see: Liskov substitution principle)

Must Have: 6, *Nice to Have:* 20, *Not Important:* 4

cppapi.class.postconditions

As a C++ API Developer
In order to Maintain a class hierarchy
I want to Ensure overriding functions meet their base class postconditions when their base class preconditions are met (see: Liskov substitution principle)

Must Have: 8, *Nice to Have:* 20, *Not Important:* 2

cppapi.class.variability

As a C++ API Developer
In order to Maintain a class hierarchy.
I want to Allow overriding functions to have narrower preconditions/wider postconditions if I want to

Must Have: 3, *Nice to Have:* 14, *Not Important:* 13

P4810 did not allow any variation in contract conditions through overrides of a virtual function, though this could be roughly accomplished with assertions in function bodies instead.

In general, the pre and postconditions of the interface you call through should be checked to be sure you're meeting the requirements you want to meet. Similarly, the concrete type's conditions should be checked to be sure it is behaving properly. Doing both checks if they are different might be an acceptable solution to allow flexibility without any increase in risk, at the possible cost of some performance in checked builds.

api.class.publicinterface

As a C++ API Developer
In order to Express public class invariants
I want to Express a restriction on the public interface of a type that all callers of the type can depend upon: can mention only public members, and is checked on entry and exit from this type's code

Must Have: 2, *Nice to Have:* 19, *Not Important:* 9

Requirements on an interface to a function should be impossible when entering into a type's functions, but there might be a desire to distinguish that so that checking is done differently when a type calls its own functions.

api.class.publicinvariants

As a C++ API Developer
In order to Express public class invariants
I want to Check invariants before and after every public method (when called from outside the type, not when one member function calls another)

Must Have: 4, *Nice to Have:* 21, *Not Important:* 5

General class invariants should be checked for validity before and after any call to a public member function of a class. Calls to friend functions need to be considered as well, as they might benefit from invalidating/validating invariants. Similarly, invariants might be checked for all objects of the type that are being touched whenever a public function is starting or finishing.

api.class.publiccalls

As a C++ API Developer
In order to Express public class invariants
I want to Check invariants before and after calling functions that are not part of this type (including virtual calls)

Must Have: 2, *Nice to Have:* 20, *Not Important:* 8

Calling from a member function to a function of another type (or a possible subtype) might require verifying that the object's invariants hold prior to calling the function and that they still hold when the other function returns.

api.class.baseinterface

As a C++ API Developer
In order to Express base class invariants
I want to Express a restriction on the protected interface of a type that derived types can depend upon: can mention only protected and public members, and is checked on entry and exit from this type's code

Must Have: 1, *Nice to Have:* 15, *Not Important:* 14

Similar to the public API of a class, the protected API is exposed to subclasses and invariants of that API should be checked whenever the boundary might be crossed.

api.class.baseinvariants

As a C++ API Developer
In order to Express base class invariants
I want to Check invariants on entry and exit of every protected method (when called from the derived type, not when one base member function calls another)

Must Have: 2, *Nice to Have:* 13, *Not Important:* 14 , *No Answer:* 1

api.class.basecalls

As a C++ API Developer
In order to Express base class invariants
I want to Check invariants before and after every call to a virtual function (when calling to the derived type)

Must Have: 2, *Nice to Have:* 18, *Not Important:* 10

api.class.privateinterface

As a C++ API Developer
In order to Express private class invariants
I want to Express an internal restriction on the private implementation of a type, can mention any member, and is checked on entry and exit from this type's code

Must Have: 4, *Nice to Have:* 14, *Not Important:* 12

api.class.privateinvariants

As a C++ API Developer
In order to Express private class invariants
I want to Check invariants on entry and exit of every public method (when called from outside the type, not when one member function calls another)

Must Have: 4, *Nice to Have:* 17, *Not Important:* 9

api.class.privatecalls

As a C++ API Developer
In order to Express private class invariants
I want to Check invariants before and after calling functions that are not part of this type (including virtual calls)

Must Have: 2, *Nice to Have:* 15, *Not Important:* 13

api.class.testing

As a C++ API Developer
In order to Test my classes
I want to For every member or friend function in my class, run my unit test framework with checking enabled for every assertion at the point where it is written, and check every postcondition at every non-exceptional exit, and test my class invariants on entry and exit from this type's code

Must Have: 11, *Nice to Have:* 9, *Not Important:* 10

cppapi.contracts.async

As a C++ API Developer
In order to Enforce contracts in async code
I want to Express contracts on callbacks such as std::function, function pointers, or references to functions, lambdas, or function objects

Must Have: 5, *Nice to Have:* 21, *Not Important:* 4

In general, any callback based solution exposes functions that will be called that might benefit from having information about their contracts also transmitted.

This might be implemented by incorporating contracts deeply into the type system, or in a more limitted way by just facilitating library types like a function with contracts.

cppapi.contracts.exception

As a C++ API Developer
In order to Enforce contracts in exception safe code

I want to Express contracts on exceptional exit

Must Have: 6, *Nice to Have:* 13, *Not Important:* 11

Generally, postconditions have been assumed to hold when a function returns normally. Expressing conditions that will hold if a function returns through an exception, or both, could also be useful.

cppapi.variadic

As a C++ API Developer
In order to Use contracts with variadic templates
I want to Allow predicate (fold) expansion

Must Have: 11, *Nice to Have:* 15, *Not Important:* 4

Fold expressions using && might be useful for a predicate, but they then lose any information about which particular argument might have violated the condition. Enabling more useful information to narrow that down would be helpful.

api.coroutines

As a C++ API Developer
In order to Use coroutines
I want to Define and check pre and post conditions as I would a regular function

Must Have: 11, *Nice to Have:* 15, *Not Important:* 4

Like regular functions, pre and postconditions on coroutines should have similar semantics.

api.coroutines.invariants

As a C++ API Developer
In order to Use coroutines
I want to Define and check invariants over all entry and exit points from a coroutine (to its awaiter or promise)

Must Have: 4, *Nice to Have:* 19, *Not Important:* 7

Similar to a class, the state of a coroutine, or what it expects of the state of the world as it suspends and resumes, might be stated as expected invariants that could be checked whenever entering and exiting.

int.conform.violation

As a Integration Developer
In order to Conform to a contract
I want to Be informed any time an interface's contract is violated

Must Have: 16, *Nice to Have:* 12, *Not Important:* 2

int.conform.postconditions

As a Integration Developer
In order to Conform to a contract
I want to Verify results from a call are expected output values

Must Have: 17, *Nice to Have:* 9, *Not Important:* 4

int.build.headeronly

As a Integration Developer
In order to Build multiple libraries
I want to Use contract-enabled header-only libraries

Must Have: 24, *Nice to Have:* 6, *Not Important:* 0

int.build.binaries

As a Integration Developer
In order to Build multiple libraries
I want to Use contract-enabled binary libraries

Must Have: 21, *Nice to Have:* 7, *Not Important:* 1 , *No Answer:* 1

int.build.binarycounts

As a Integration Developer
In order to Build multiple libraries
I want to Only be required to manage a small, common set of build/link configurations

Must Have: 9, *Nice to Have:* 15, *Not Important:* 6

int.build.control

As a Integration Developer
In order to Debug multiple libraries
I want to Enable checks only within a selected library

Must Have: 15, *Nice to Have:* 14, *Not Important:* 1

int.build.control2

As a Integration Developer
In order to Debug multiple libraries
I want to Enable checks on multiple libraries simultaneously

Must Have: 14, *Nice to Have:* 14, *Not Important:* 2

int.debug.callsites

As a Integration Developer
In order to Debug multiple call sites
I want to Enable checks only on selected call sites

Must Have: 7, *Nice to Have:* 18, *Not Important:* 5

Often it is helpful to limit what gets enabled to functions called from some subset of a program, and the rest of the program might not perform acceptably enough to test if hose same functions are always checked.

int.violations.information

As a Integration Developer
In order to Correct failed checks
I want to Be informed what check failed, when, where, and how

Must Have: 20, *Nice to Have:* 9, *Not Important:* 1

int.violations.transmit

As a Integration Developer
In order to Correct failed checks
I want to Transmit check failure information in environment-specific ways (logs, email, special hardware traps, popup windows, blazing sirens, etc).

Must Have: 11, *Nice to Have:* 14, *Not Important:* 5

int.violations.custom

As a Integration Developer
In order to Correct failed checks
I want to Install custom violation handler where I can inject custom logic to trap errors

Must Have: 14, *Nice to Have:* 9, *Not Important:* 7

int.violations.common

As a Integration Developer
In order to Unify violation handling
I want to Be able to override how library violations are handled in the combined software to point into my handling code

Must Have: 15, *Nice to Have:* 9, *Not Important:* 6

int.violations.override

As a Integration Developer
In order to Be independent of build environment
I want to Be able to define and override violation handler via source code

Must Have: 4, *Nice to Have:* 11, *Not Important:* 14 , *No Answer:* 1

int.build.minimize

As a Integration Developer
In order to Minimize checking overhead
I want to Disable library postconditions, asserts, and invariants, without disabling library preconditions (assuming the library is tested and stable and my code is not)

Must Have: 8, *Nice to Have:* 14, *Not Important:* 8

int.control.build

As a Integrated Software Provider
In order to Ensure the combined software is correct
I want to At build time, turn on and off what checking happens.

Must Have: 21, *Nice to Have:* 6, *Not Important:* 3

int.control.runtime

As a Integrated Software Provider
In order to At runtime, control what checking happens.
I want to Turn checks on at run time

Must Have: 5, *Nice to Have:* 11, *Not Important:* 14

int.conrol.subsets.build

As a Integrated Software Provider
In order to Ensure the combined software is correct
I want to Turn on any subset of individual (call site) checks on at build time

Must Have: 9, *Nice to Have:* 12, *Not Important:* 8 , *No Answer:* 1

int.control.subsets.runtime

As a Integrated Software Provider
In order to Ensure the combined software is correct
I want to Turn on any subset of individual (call site) checks on at run time

Must Have: 3, *Nice to Have:* 12, *Not Important:* 15

int.consistency

As a Integrated Software Provider
In order to Ensure the combined software is correct
I want to Verify all annotations are globally consistent when integrated

Must Have: 3, *Nice to Have:* 20, *Not Important:* 6 , *No Answer:* 1

int.control.subsets

As a Integrated Software Provider
In order to Ensure individual features are correct
I want to Have a way to audit (named or semantic) subsets of checks for various deployments

Must Have: 4, *Nice to Have:* 16, *Not Important:* 9 , *No Answer:* 1

int.build.common

As a Integrated Software Provider
In order to Manage binary delivery
I want to Be able to use the same executable regardless of contract enforcement mode

Must Have: 4, *Nice to Have:* 10, *Not Important:* 16

int.testing.control

As a Integrated Software Provider
In order to Define "Code Under Test"
I want to Selectively enable checking for a set of functions which could name either an individual function or an overload set

Must Have: 3, *Nice to Have:* 16, *Not Important:* 11

int.testing.controltypes

As a Integrated Software Provider
In order to Define "Code Under Test"
I want to Selectively enable checking for a set of types and all their members

Must Have: 3, *Nice to Have:* 15, *Not Important:* 12

int.testing.transitivity

As a Integrated Software Provider
In order to Define "Code Under Test"
I want to Selectively enable checking for a set of types and all their transitively nested types and members

Must Have: 3, *Nice to Have:* 14, *Not Important:* 13

int.testing.modules

As a Integrated Software Provider
In order to Define "Code Under Test"
I want to Selectively enable checking for a translation unit or module and all (non transitive) types and functions within

Must Have: 5, *Nice to Have:* 19, *Not Important:* 6

int.build.unchecked

As a Integrated Software Provider
In order to Test final deliverable
I want to Turn off build time checking to remove checking overhead

Must Have: 15, *Nice to Have:* 6, *Not Important:* 9

int.runtime.unchecked

As a Integrated Software Provider
In order to Test final deliverable
I want to Turn off run time checking to remove checking overhead

Must Have: 22, *Nice to Have:* 5, *Not Important:* 3

int.build.optimize

As a Integrated Software Provider
In order to Test final deliverable
I want to Turn on run time optimization to leverage annotation assumptions

Must Have: 12, *Nice to Have:* 12, *Not Important:* 6

cpplib.headeronly

As a C++ Library Developer
In order to Use templates
I want to Be able to ship header only library

Must Have: 25, *Nice to Have:* 4, *Not Important:* 1

cpplib.insulation

As a C++ Library Developer
In order to Control the tradeoff between need for client recompilation and contract condition visibility
I want to Insulate contract conditions with the function definition, or insulate only the definition while putting contract conditions on a redeclaration - visible to static analysis tools in all TUs.

Must Have: 6, *Nice to Have:* 13, *Not Important:* 11

Contract conditions on only the first declaration (as proposed by P4810) mean that the condition clutters the readable interface and changes in a contract condition force client recompilation. Contract conditions not in the header file are not going to be visible to tools that are attempting to limit what they consume to a single translation unit. Combined, these result in a desire to sometimes put the contracts with the implementing definition (in a .cpp file) and sometimes put them in a redeclaration in a header file (where one might also put inline function definitions).

For free functions this would just require relaxing the requirements on where contracts are. For member functions, allowing member function redeclaration would be needed to put the contracts in a less obtrusive place that is still visible outside of the defining TU. See P1320 for more discussion of this.

lib.maintenance.noconfig

As a Library Provider
In order to Simplify maintenance
I want to Not require extra build steps to be documented

Must Have: 6, *Nice to Have:* 10, *Not Important:* 14

lib.maintenance.nowhining

As a Library Provider
In order to Simplify maintenance
I want to Not have users complain about my product due to modifications of annotations resulting from their build configuration

Must Have: 4, *Nice to Have:* 9, *Not Important:* 17

lib.integration.noconfig

As a Library Provider
In order to Support successful integration
I want to Not require extra build steps to be learned or performed

Must Have: 4, *Nice to Have:* 13, *Not Important:* 13

lib.integration.nowhining

As a Library Provider
In order to Support successful integration
I want to Not have my users accidentally modify my careful annotations

Must Have: 7, *Nice to Have:* 11, *Not Important:* 12

arch.nomacros

As a Technical Architect
In order to Maintain quality of code base
I want to Express assertions in a way that does not rely on C macros (i.e., there is no valid technical reason for a programmer not to use the new way, including space, time, tooling, and usability/complexity reasons, compared to C's assert macro)

Must Have: 10, *Nice to Have:* 13, *Not Important:* 7

arch.complete

As a Technical Architect
In order to Have a consistent and holistic contracts facility
I want to Specify preconditions/postconditions/assertions/invariants that express my expectations about the expected valid state of my program in the form of compilable boolean expressions, that can be checked statically or dynamically (as opposed to disjointed state where these features are factored into bits)

Must Have: 14, *Nice to Have:* 12, *Not Important:* 3 , *No Answer:* 1

hardware.performance

As a Hardware Architect
In order to Improve system-level performance
I want to Be able to design new hardware + optimizations, carefully dovetailed into one another, that depend on statically-unprovable facts being annotated in the code

Must Have: 10, *Nice to Have:* 10, *Not Important:* 8 , *No Answer:* 2

sdev.bestpractices

As a Senior Developer
In order to Set an example
I want to Demonstrate best practice in defensive programming

Must Have: 18, *Nice to Have:* 9, *Not Important:* 3

sdev.quality

As a Senior Developer
In order to Enforce code quality
I want to Discourage reliance on observable out-of-contract behavior by causing check failure to hard stop program or build

Must Have: 16, *Nice to Have:* 6, *Not Important:* 8

sdev.maturity

As a Senior Developer
In order to Enforce mature, finalized contracts
I want to Disable continuation on violation of stable and correct individual contracts

Must Have: 19, *Nice to Have:* 7, *Not Important:* 4

sdev.control

As a Senior Developer
In order to Enforce mature, finalized contracts
I want to Disable remapping of semantics on stable and correct individual contracts

Must Have: 9, *Nice to Have:* 12, *Not Important:* 9

jdev.understand.contracts

As a Junior Developer
In order to Understand the API

I want to A uniform, fluent description of expected input values, expected output values, side effects, and all logical pre and post conditions

Must Have: 14, *Nice to Have:* 15, *Not Important:* 1

jdev.understand.violations

As a Junior Developer
In order to Understand the API
I want to Be informed when my usage is out of contract

Must Have: 21, *Nice to Have:* 9, *Not Important:* 0

jdev.understand.buildfailures

As a Junior Developer
In order to Understand the program
I want to Know why my software is not building

Must Have: 21, *Nice to Have:* 6, *Not Important:* 3

jdev.understand.aborting

As a Junior Developer
In order to Understand the program
I want to Know why my software is aborting

Must Have: 20, *Nice to Have:* 9, *Not Important:* 1

jdev.understand.omniscience

As a Junior Developer
In order to Understand the program
I want to Know why my software is out of contract

Must Have: 18, *Nice to Have:* 11, *Not Important:* 1

jdev.understand.buildviolation

As a Junior Developer
In order to Understand the program
I want to Know that my program or build was halted due to contract violation

Must Have: 23, *Nice to Have:* 7, *Not Important:* 0

jdev.understand.all

As a Junior Developer
In order to Understand the facility
I want to Be able to build a program with contracts after reasonably short tutorial

Must Have: 14, *Nice to Have:* 12, *Not Important:* 4

jdev.understand.keywords

As a Junior Developer
In order to Understand the facility
I want to Have keywords with precise and unambiguous meanings

Must Have: 15, *Nice to Have:* 13, *Not Important:* 2

jdev.bestpractices

As a Junior Developer
In order to Improve my code
I want to Learn about software best practices by example

Must Have: 14, *Nice to Have:* 12, *Not Important:* 4

adev.fast

As a Agile Developer
In order to Iterate quickly
I want to Be able to write and modify contracts quickly without heavy boiler plate or up front cost

Must Have: 15, *Nice to Have:* 14, *Not Important:* 1

adev.evolve

As a Agile Developer
In order to Safeguard evolving code
I want to Assert against conditions I am aware of but not finished handling fully

Must Have: 10, *Nice to Have:* 12, *Not Important:* 8

bdev.confidentiality

As a Business Developer
In order to Maintain confidentiality
I want to Not expose diagnostic information (source location, expressions, etc.) in the software I deliver to clients, even when I choose to have contracts enforced in the software I deliver

Must Have: 5, *Nice to Have:* 11, *Not Important:* 14

pdev.speed

As a Performance Sensitive Developer
In order to Enable better performance
I want to Annotate my code with assumptions, likelihoods, or reachability information that a tool might not be able to deduce, but that I would be confident of

Must Have: 13, *Nice to Have:* 10, *Not Important:* 7

pdev.morespeed

As a Performance Sensitive Developer
In order to Enable better performance
I want to Be able to give statically-unprovable facts to current and novel optimizers in terms of semantics my program does not depend-on but optimizers can't figure out

Must Have: 13, *Nice to Have:* 11, *Not Important:* 6

pdev.footgun

As a Performance Sensitive Developer
In order to Enable better performance
I want to Accept responsibility for a malformed program that might result from eventually false information given by my annotations

Must Have: 15, *Nice to Have:* 8, *Not Important:* 7

pdev.safety.isolation

As a Performance Sensitive Developer
In order to Have safety critical paths
I want to Isolate safety checks from performance annotations

Must Have: 12, *Nice to Have:* 11, *Not Important:* 7

pdev.safety.critical

As a Performance Sensitive Developer
In order to Have safety critical paths
I want to Retain checking even when optimizing with performance annotations

Must Have: 14, *Nice to Have:* 8, *Not Important:* 8

qdev.checkall

As a Quality Sensitive Developer
In order to Enable full checking
I want to Ensure all checks (pre, post, assert, invariant) are enabled

Must Have: 18, *Nice to Have:* 10, *Not Important:* 2

qdev.correctness

As a Quality Sensitive Developer
In order to Validate correctness
I want to Signify the predicates that should be verified by an analysis tool

Must Have: 5, *Nice to Have:* 19, *Not Important:* 6

qdev.tooling

As a Quality Sensitive Developer
In order to Manage multiple tools
I want to Signify subset of individual annotations to be consumed by a specific kind of verification tool

Must Have: 2, *Nice to Have:* 14, *Not Important:* 14

qdev.tooling.control

As a Quality Sensitive Developer
In order to Manage multiple tools
I want to Signify subset of individual annotations to be consumed by a specific instance of verification tool

Must Have: 1, *Nice to Have:* 15, *Not Important:* 14

qdev.tooling.undefined

As a Quality Sensitive Developer
In order to Manage multiple tools
I want to Use predicates that may not be understood by all instances of verification

Must Have: 9, *Nice to Have:* 12, *Not Important:* 9

qdev.tooling.undefinedkinds

As a Quality Sensitive Developer
In order to Manage multiple tools
I want to Use predicates that may not be understood by all kinds of verification

Must Have: 10, *Nice to Have:* 12, *Not Important:* 8

qdev.tooling.behavior

As a Quality Sensitive Developer
In order to Manage multiple tools
I want to Integrate the results of that static checker into how my program behaves in different ways: assume proven predicates, make unprovable predicates ill- formed, etc.

Must Have: 4, *Nice to Have:* 11, *Not Important:* 15

qdev.testing

As a Quality Sensitive Developer
In order to Unit test predicates
I want to Override failure handler to trigger test failure instead of termination

Must Have: 13, *Nice to Have:* 9, *Not Important:* 8

qdev.handler.testing

As a Quality Sensitive Developer
In order to Unit test violation handlers
I want to Have a way to run handler on all combinations of available build modes

Must Have: 6, *Nice to Have:* 13, *Not Important:* 11

qdev.fuzz.testing

As a Quality Sensitive Developer
In order to Catch unexpected failure modes
I want to Log all predicate failure during fuzz testing

Must Have: 11, *Nice to Have:* 14, *Not Important:* 5

crit.control

As a Critical Software Developer
In order to Have a verifiable release system
I want to Be able to control the configuration of contracts from a central point

Must Have: 11, *Nice to Have:* 15, *Not Important:* 4

crit.noundef

As a Critical Software Developer
In order to Avoid undefined behavior
I want to Have contract violation at run-time always have well-defined behavior

Must Have: 9, *Nice to Have:* 4, *Not Important:* 17

crit.recovery

As a Critical Software Developer
In order to Not have a faulty program lead to catastrophic failure
I want to Have access to a recovery path after contract violation

Must Have: 7, *Nice to Have:* 8, *Not Important:* 15

crit.redundancy

As a Critical Software Developer
In order to Not have a faulty program lead to catastrophic failure
I want to Be able to express error handling that may be redundant with contract checking

Must Have: 11, *Nice to Have:* 8, *Not Important:* 11

crit.interaction

As a Critical Software Developer
In order to Not have a faulty program lead to catastrophic failure
I want to Not have contract build or run modes possibly be able to change or disable related error handling in any way

Must Have: 12, *Nice to Have:* 5, *Not Important:* 13

crit.locality

As a Critical Software Developer
In order to Be assured a critical violation uses a critical recovery path
I want to Couple recovery path to a specific contract within the source

Must Have: 5, *Nice to Have:* 6, *Not Important:* 19

crit.testing

As a Critical Software Developer
In order to Meet code coverage requirements
I want to Be able to run both success and failure branches in my test environment

Must Have: 11, *Nice to Have:* 11, *Not Important:* 8

crit.production.checking

As a Critical Software Developer
In order to Have redundant layering
I want to Be able to continue to run checks in a production environment (even after formal testing is complete)

Must Have: 15, *Nice to Have:* 8, *Not Important:* 6 , *No Answer:* 1

crit.more.coverage

As a Critical Software Developer
In order to Maximize coverage
I want to Be able to run checks in a production environment that are considered "cheap" compared to the expected cost of entering an invalid state

Must Have: 11, *Nice to Have:* 13, *Not Important:* 6

crit.noassume

As a Critical Software Developer
In order to Avoid unexpected or undefined behavior
I want to Ensure checks will never be __assume'd/__builtin_assume'd by the compiler as if they were facts injected into the program (otherwise, if such an assumption ever failed, I would be running a different program that is not equivalent to the one I wrote; assumptions can expand the set of possible executions by injecting facts not otherwise knowable to the compiler)

Must Have: 12, *Nice to Have:* 5, *Not Important:* 13

sec.noattacks

As a Security Sensitive Developer
In order to Limit attack vectors
I want to Be unable to insert code paths (eg. violation handlers) at run time (eg. build time only)

Must Have: 12, *Nice to Have:* 9, *Not Important:* 8 , *No Answer:* 1

sec.certify

As a Security Sensitive Developer
In order to Deliver a certified product
I want to Have build tool only link to a preapproved violation handler

Must Have: 3, *Nice to Have:* 15, *Not Important:* 12

analysis.runtime

As a User of Analysis Tools
In order to Improve runtime correctness
I want to Have runtime checks generated by the tool

Must Have: 7, *Nice to Have:* 14, *Not Important:* 8 , *No Answer:* 1

analysis.optimization

As a User of Analysis Tools
In order to Improve runtime performance
I want to Have runtime optimizations generated by the tool

Must Have: 5, *Nice to Have:* 17, *Not Important:* 7 , *No Answer:* 1

analysis.symbolic

As a User of Analysis Tools
In order to Allow symbolic analysis
I want to Have symbolic proofs for soundness and consistency performed before compile time

Must Have: 4, *Nice to Have:* 20, *Not Important:* 5 , *No Answer:* 1

analysis.compiletime

As a User of Analysis Tools
In order to Allow code analysis
I want to Have code source, AST, or instruction inspection during compile time

Must Have: 4, *Nice to Have:* 17, *Not Important:* 8 , *No Answer:* 1

analysis.binaries

As a User of Analysis Tools
In order to Allow binary analysis
I want to Have binary inspection after compile time

Must Have: 2, *Nice to Have:* 10, *Not Important:* 17 , *No Answer:* 1

analysis.information

As a User of Analysis Tools
In order to Improve the quality of analysis
I want to Be able to hint to the analyzer information it may be unable to deduce from source code alone (eg. *5 / opaque(); [[opaque() != 0]]*)

Must Have: 6, *Nice to Have:* 17, *Not Important:* 6 , *No Answer:* 1

analysis.legacy

As a Provider of Analysis Tools
In order to Extend my existing engine
I want to Be able to map pre-existing contract features in tools to a standardized language syntax

Must Have: 5, *Nice to Have:* 10, *Not Important:* 14 , *No Answer:* 1

teach.bestpractices

As a Teacher
In order to Demonstrate best practice
I want to Be able to express defensive programming, programming by contract, and test driven development to introductory students

Must Have: 15, *Nice to Have:* 13, *Not Important:* 2

teach.standardized

As a Teacher
In order to Demonstrate best practice
I want to Not rely on custom libraries or proprietary extensions

Must Have: 15, *Nice to Have:* 12, *Not Important:* 3

teach.lifecycle

As a Teacher
In order to Demonstrate best practice
I want to Demonstrate mock lifecycle by switching simple compiler flags to control which checks are enabled

Must Have: 6, *Nice to Have:* 19, *Not Important:* 5

teach.portable

As a Teacher
In order to Manage many students
I want to Have examples compilable by a standard compiler on any system

Must Have: 23, *Nice to Have:* 3, *Not Important:* 4

teach.dumbstudents

As a Teacher
In order to Manage many students
I want to Have examples that are easy to build without digression into build systems

Must Have: 15, *Nice to Have:* 9, *Not Important:* 6

teach.teachable

As a Teacher
In order to Build layers of understanding
I want to Have simple explanation of assertions and their use to support simple programming tasks, including debugging erroneous programs.

Must Have: 18, *Nice to Have:* 9, *Not Important:* 3

teach.layering

As a Teacher
In order to Build layers of understanding
I want to Support the ability for advanced uses of contracts to be distributed across many different courses in a C++-focused computer science curriculum.

Must Have: 9, *Nice to Have:* 15, *Not Important:* 6

compiler.benice

As a Compiler Developer
In order to Deliver best experience to my customers
I want to Maximize implementation freedom by limiting what is strictly required by the standard

Must Have: 6, *Nice to Have:* 12, *Not Important:* 12

compiler.best

As a Compiler Developer
In order to Deliver the best implementation
I want to Have a clear and simple specification that meets clear need

Must Have: 19, *Nice to Have:* 10, *Not Important:* 1

large.complex

As a Large Codebase Developer
In order to Debug complex issues
I want to Have composable and fine grained control over which checks are run, without requiring source code changes. Specifically the checks for only one function or some grouping of functions

Must Have: 11, *Nice to Have:* 14, *Not Important:* 5

large.critical

As a Large Codebase Developer
In order to Enable/Disable checking on critical/hot paths
I want to Control whether checks are run based on where they are being called from

Must Have: 8, *Nice to Have:* 13, *Not Important:* 9

large.modernize

As a Large Codebase Owner
In order to Modernize my code base
I want to Introduce standardized contracts to replace my macro-based contracts

Must Have: 17, *Nice to Have:* 12, *Not Important:* 1

large.stillmacros

As a Large Codebase Owner
In order to Modernize my code base
I want to Have my existing macro-based facilities interoperate smoothly with standardized contracts so I can do the migration gradually

Must Have: 9, *Nice to Have:* 16, *Not Important:* 5

large.observation

As a Large Codebase Owner
In order to Introduce new contracts into an existing system
I want to Have failed individual checks from existing code optionally warn instead of hard stop

Must Have: 14, *Nice to Have:* 9, *Not Important:* 7

large.introduction

As a Large Codebase Owner
In order to Introduce new contracts into an existing system
I want to Have failed checks from a new library optionally warn instead of hard stop

Must Have: 14, *Nice to Have:* 9, *Not Important:* 7

large.separability

As a Large Codebase Owner
In order to Introduce new parameters or invariants into a contracts based system
I want to Be able to include distinct clauses for each parameter or invariant with their own individual failure or build controls

Must Have: 5, *Nice to Have:* 8, *Not Important:* 17

large.newenvironment

As a Large Codebase Owner
In order to Introduce new elements into a contracts based system
I want to Have failed checks caused by a change in environment optionally warn instead of hard stop

Must Have: 7, *Nice to Have:* 13, *Not Important:* 10

large.newcompiler

As a Large Codebase Owner
In order to Introduce new elements into a contracts based system
I want to Have failed checks caused by a change in compiler optionally warn instead of hard stop

Must Have: 5, *Nice to Have:* 15, *Not Important:* 10

large.nogoingback

As a Large Codebase Owner
In order to Prevent regressions
I want to Have trusted contracts fail fast and hard stop

Must Have: 17, *Nice to Have:* 11, *Not Important:* 2

large.scalability

As a Large Codebase Owner
In order to Scale violation handling
I want to Be able to log violations in my organization specific format

Must Have: 14, *Nice to Have:* 11, *Not Important:* 5

large.simulation.disable

As a Large Codebase Owner
In order to Allow simulation or post-mortem testing of known failure modes
I want to Optionally disable checking on a subset of individual annotations

Must Have: 10, *Nice to Have:* 10, *Not Important:* 10

large.simulation.enable

As a Large Codebase Owner
In order to Allow simulation or post-mortem testing of known failure modes
I want to Optionally allow checking of a subset of individual annotations to fail and access its recovery path

Must Have: 8, *Nice to Have:* 9, *Not Important:* 13

large.simulation.ignore

As a Large Codebase Owner
In order to Allow simulation or post-mortem testing of known failure modes
I want to Optionally allow checking of a subset of individual annotations to fail and continue failing

Must Have: 7, *Nice to Have:* 9, *Not Important:* 14

large.perfcontrol.build

As a Large Codebase Owner
In order to Manage performance cost
I want to Constrain the set of built time checks according to their performance overhead

Must Have: 5, *Nice to Have:* 12, *Not Important:* 13

large.perfcontrol.runtime

As a Large Codebase Owner
In order to Manage performance cost
I want to Constrain the set of runtime checks according to their performance overhead

Must Have: 13, *Nice to Have:* 11, *Not Important:* 6

large.narrowing

As a Large Codebase Owner
In order to Tune contract width in complex system
I want to Be able to narrow individual contract so it fails in testing not in production

Must Have: 6, *Nice to Have:* 13, *Not Important:* 11

embedded.nochecking

As a Small Machine Developer
In order to Minimize executable footprint
I want to Remove all checking and diagnostic (eg. source location) overhead entirely from the final binary

Must Have: 19, *Nice to Have:* 7, *Not Important:* 4

embedded.nologging

As a Small Machine Developer
In order to Minimize executable footprint
I want to Remove all logging and diagnostic (but not checking) overhead from the final binary

Must Have: 11, *Nice to Have:* 14, *Not Important:* 4 , *No Answer:* 1

embedded.minimize

As a Small Machine Developer
In order to Minimize executable footprint
I want to Remove all but the most important diagnostic overhead from the final binary

Must Have: 9, *Nice to Have:* 14, *Not Important:* 7

wg21.everythingelse

As a Language Developer
In order to Interoperate with Contracts
I want to Have a clear way to understand how contracts will interact with the standard library

Must Have: 14, *Nice to Have:* 11, *Not Important:* 4 , *No Answer:* 1

wg21.otherfeatures

As a Language Developer
In order to Extend contracts beyond pre/post conditions on functions
I want to Be able to use contract-like syntax on past or present runtime checkable language features such as switches, pattern matching, etc. or what might happen on signed integer overflow, etc. This might allow configuration of trapping, logging, or assuming in other areas of language UB.

Must Have: 4, *Nice to Have:* 14, *Not Important:* 12

Acknowledgements

Thanks to everyone who has participated in the SG21 reflector, telecon, and our first meeting in Belfast. Herb Sutter contributed greatly by taking the vast raw data that made up our use cases and painfully populated the online survey that everyone participated in. Thanks also to all 30 respondents who took a significant portion of their personal time to read, understand, and express their opinions on the 195 use cases presented here.